

推导完全分类

我会把两个 bit-exact 改动分开裁决：区间套证书属于定义语义，增量塔属于与全量 canonical 等价的前沿重算问题。重点是分别给出数学判定、修复条件和验收边界。

下面给出严格裁决。两个问题性质不同：

问题①是区间套证书定义层错误：端点相等应改为区间包含。

问题②是增量实现的 frontier 语义错误：未确认后缀必须重算。

如果目标是检验“缠师 100% 区间套判据 + 完整多级结构”的 alpha，那么这两个都必须修。否则回测对象仍然是简化策略，不是完整缠论。

一、问题①：端点相等 vs 区间包含

1. 先定义区间套证书

你的形式化为：

$$N_{\ell \downarrow e}^{\delta}(x_t) \in \{0, 1\}.$$

其中：

- $\delta \in \{+1, -1\}$ 是交易方向；
- ℓ 是起始级别；
- e 是执行级别；
- J_k^{δ} 是第 k 级定位区间。

基例：

$$N_{e \downarrow e}^{\delta} = Conf_e^{\delta}.$$

其中：

$$Conf_e^{+} = B_{1,e} \vee B_{2,e} \vee B_{3,e},$$

$$Conf_e^{-} = S_{1,e} \vee S_{2,e} \vee S_{3,e}.$$

递归步：

$$N_{k \downarrow e}^{\delta} = Cand_k^{\delta} \wedge [J_{k-1}^{\delta} \subseteq J_k^{\delta}] \wedge N_{k-1 \downarrow e}^{\delta}, \quad k > e.$$

所以多级区间套的本体是：

$$J_e^{\delta} \subseteq J_{e+1}^{\delta} \subseteq \cdots \subseteq J_{\ell}^{\delta}.$$

这是区间包含链，不是端点相等链。

2. 当前实现的端点相等条件过强

当前实现的关键行是：

$$target_idx = \text{position}\{m : m.end_index = source_index\}.$$

这相当于要求：

$$\rho(m) = s.$$

其中：

- m 是上一级节点的某个 sub_move；
- $\rho(m)$ 是该 sub_move 的右端点；
- s 是下层信号或执行点的 source index。

这比区间套要求强得多。

区间套只需要：

$$s \in I(m)$$

或更完整地：

$$J_{k-1} \subseteq I(m).$$

端点相等要求：

$$s = \rho(m).$$

显然：

$$s = \rho(m) \Rightarrow s \in I(m),$$

但：

$$s \in I(m) \not\Rightarrow s = \rho(m).$$

因此端点相等是区间包含的真子条件。

所以当前实现实际检验的是：

$$\boxed{\text{区间套} + \text{每一级都恰好落在父级 submove 右端点}}$$

这是额外约束，原区间套没有要求。

3. 严格反例：区间套成立但端点相等失败

设第 k 级父候选区间为：

$$J_k = [0, 100].$$

其第 $k - 1$ 级某个子段为：

$$m = [20, 80].$$

执行级信号的 source index 为：

$$s = 50.$$

则：

$$s \in [20, 80],$$

并且若下级定位区间：

$$J_{k-1} = [45, 55],$$

则：

$$J_{k-1} \subseteq J_k.$$

所以区间套语义成立。

但：

$$s = 50 \neq 80 = \rho(m).$$

因此当前端点相等实现拒绝该证书。

所以：

端点相等会产生 false negative。

你的实测 $\text{depth} \geq 2$ 为 0，正是这种 false negative 的系统性表现。

4. 原文语义裁决：rung 应是区间包含，不是端点相等

区间套的语义是：

从大级别背驰段逐级收缩到小级别背驰段，最后定位到执行级买卖点。

这要求小级别定位对象位于大级别定位窗口内部：

$$J_{k-1} \subseteq J_k.$$

它不要求：

$$\rho(J_{k-1}) = \rho(J_k)$$

更不要求：

$$\text{source_index} = \rho(m).$$

所以裁决：

端点相等可用于“买卖点挂靠本级走势段右端点”，

但不能用于：

跨级区间套 rung 定位。

跨级 rung 的正确语义是：

子区间被父区间包含。

二、如何改：不是简单 point contains, 而是 child-interval contains

你提出“ $\text{start} \leq \text{source} \leq \text{end}$ ”是方向对的，但还不够严格。

因为递归定义要求：

$$J_{k-1} \subseteq J_k,$$

不是只要求：

$$\text{source_index} \in J_k.$$

所以最严格实现应当是 bottom-up：

先确定低一级 J_{k-1} , 再在高一级候选中找包含 J_{k-1} 的 J_k .

定义第 k 级候选集合：

$$\mathcal{C}_k^\delta(t) = \{c : \text{DivCand}_k^\delta(c, t) = 1\}.$$

给定低一级定位区间 J_{k-1} , 定义可承接父候选集合：

$$\mathcal{C}_k^\delta(J_{k-1}, t) = \{c \in \mathcal{C}_k^\delta(t) : J_{k-1} \subseteq I(c)\}.$$

然后：

$$\text{Cand}_k^\delta(J_{k-1}, t) := [\mathcal{C}_k^\delta(J_{k-1}, t) \neq \emptyset].$$

若非空，用确定性选择器：

$$c_k = \text{Sel}_\Theta(\mathcal{C}_k^\delta(J_{k-1}, t)).$$

再定义：

$$J_k^\delta = I(c_k).$$

于是递归变成：

$$N_{k \downarrow e}^\delta = \text{Cand}_k^\delta(J_{k-1}, t) \wedge N_{k-1 \downarrow e}^\delta.$$

其中包含关系已经内置在：

$$\mathcal{C}_k^\delta(J_{k-1}, t)$$

里。

这比“先全局选一个 J_k , 再检查 $J_{k-1} \subseteq J_k$ ”更安全。

为什么不能先全局 Sel, 再检查包含？

反例：

第 k 级有两个候选：

$$c_1 = [0, 10], \quad c_2 = [0, 100].$$

低一级区间：

$$J_{k-1} = [50, 60].$$

若全局选择器按较早 end time 选：

$$Sel(c_1, c_2) = c_1,$$

则：

$$J_{k-1} \not\subseteq c_1.$$

于是证书失败。

但实际上：

$$J_{k-1} \subseteq c_2.$$

存在合法父候选。

所以若 `Sel` 无条件化于 child interval，会制造 false negative。

因此正确选择器必须是：

Sel_Θ 作用在“包含 child interval 的候选集合”上。

三、区间包含后的唯一性证明

担心：

source 可能落在多个上级段区间内。

这是对的。解决方式有两层。

1. 若塔是良式分解，直接唯一

定义良式直接子段分解。

给定父区间：

$$I(p) = [a, b].$$

其直接子段为：

$$m_1, \dots, m_r.$$

采用半开或右闭唯一归属约定：

$$I(m_j) = (t_{j-1}, t_j]$$

其中：

$$a = t_0 < t_1 < \dots < t_r = b.$$

则对任意：

$$s \in (a, b],$$

存在唯一 j 使:

$$s \in I(m_j).$$

所以如果 sub_moves 是规范连续分解, 区间包含定位天然唯一。

2. 若有重叠候选, 用 Sel 保证唯一

若候选是候选背驰段, 不一定不重叠。则定义:

$$\mathcal{C}_k^\delta(J_{k-1}, t) = \{c : J_{k-1} \subseteq I(c)\}.$$

该集合有限。

定义一个全序:

$$\prec_\Theta.$$

例如:

$$(width, end_time, start_time, idx)$$

或你原来的:

$$(end_time, start_time, idx)$$

只要是严格全序即可。

然后:

$$\text{Sel}_\Theta(C) = \min_{\prec_\Theta} C.$$

因为 C 有限非空, 且 $\prec_\Theta$ 是全序, 所以:

$$\boxed{\exists! \text{Sel}_\Theta(C).}$$

因此唯一性成立。

四、区间套证书 soundness / completeness

定理 1: soundness

若新实现返回:

$$N_{\ell \downarrow e}^\delta = 1,$$

则存在唯一链:

$$J_e \subseteq J_{e+1} \subseteq \cdots \subseteq J_\ell.$$

证明

基例:

$$N_{e \downarrow e}^\delta = \text{Conf}_e^\delta = 1.$$

由执行级 Conf 取得 J_e 。

递归步：假设已有：

$$J_{k-1}.$$

新实现要求：

$$\mathcal{C}_k^\delta(J_{k-1}, t) \neq \emptyset.$$

所以存在：

$$c_k$$

使：

$$J_{k-1} \subseteq I(c_k).$$

由 Sel 得唯一：

$$c_k = \text{Sel}_\Theta(\mathcal{C}_k^\delta(J_{k-1}, t)).$$

定义：

$$J_k = I(c_k).$$

于是：

$$J_{k-1} \subseteq J_k.$$

逐层得到嵌套链。

证毕。

定理 2：endpoint 旧实现是新实现的真子集

若旧实现成功，则每一层都有：

$$source_index = \rho(m).$$

因此：

$$source_index \in I(m).$$

若低级 J_{k-1} 是以 source 为端点的小区间且落在 m 内，则新实现也可成功。

但存在反例：

$$source \in I(m), \quad source \neq \rho(m),$$

旧实现失败，新实现成功。

所以：

$$\boxed{\Gamma_{\text{old}} \subsetneq \Gamma_{\text{new}}}.$$

这解释了旧回测有效嵌套深度几乎全为 0。

五、问题①最终裁决

(a) 区间包含是区间套 `rung` 的正确语义；端点相等没有跨级 `rung` 的原文依据。

$\boxed{\text{\texttt{(b) 改成区间包含后, 唯一性由良式分解或 Sel_Theta 保证。}}}$

但实现不要仅仅改成：

$$start \leq source \leq end.$$

最严格应改成：

$$J_{child} \subseteq J_{parent}.$$

即：

$$start(parent) \leq start(child) \wedge end(child) \leq end(parent).$$

这才和你的 Lean 证书公式完全一致。

六、问题②：增量级别塔 vs 全量重算

现在看第二个问题。

你们的目标是：

$$\text{增量塔 } T_t^{inc} = \text{全量重算塔 } T_t^{full} \quad \forall t.$$

这是 bit-exact 的定义。

1. 定义 full tower

令历史为：

$$h_{0:t}.$$

全量重算函数：

$$F_t = F(h_{0:t}).$$

增量更新函数：

$$T_{t+1}^{inc} = U(T_t^{inc}, h_{t+1}).$$

bit-exact 要求：

$$T_t^{inc} = F_t \quad \forall t.$$

这是你的 parity 测试要证明的性质。

2. confirmed prefix / mutable frontier 分解

要使增量等价于全量，必须把塔分成：

$$F_t = P_t \oplus R_t.$$

其中：

- P_t ：confirmed prefix，未来不会被新 bar 改写；
- R_t ：mutable frontier，未来可能被新 bar 改写。

严格要求：

P_t 可复用， R_t 必须重算。

不能把未确认的最后窗口放进 P_t 。

3. 当前 bug 的形式化

当前 `detect_centers_windowed_resume` 把最后一个未确认窗口视为不可变，即把它错误纳入：

$$P_t.$$

于是增量更新为：

$$T_{t+1}^{inc} = P_t^{wrong} \oplus Detect(h_{b:t+1})$$

其中 b 晚于真正 frontier start。

全量重算为：

$$F_{t+1} = P_t^{true} \oplus Detect(h_{a:t+1})$$

其中：

$$a \leq b.$$

如果未确认窗口在新 bar 到达后被修正或删除，则：

$$P_t^{wrong} \neq P_t^{true}.$$

于是：

$$T_{t+1}^{inc} \neq F_{t+1}.$$

这正是你观测到的：

增量塔 L0 多 1 个中枢。

七、纯实现 bug 还是定义冲突？

裁决：

如果 canonical 定义是“增量必须等于全量重算”，那这是实现 bug。

因为全量重算会重新评估未确认最后窗口；增量也必须重新评估同一 mutable frontier。

没有定义冲突。

只有在你决定更换 canonical：

OnlineTower = 一旦 emit 就不可回溯

时，它才变成定义冲突。

但那会放弃：

$$T_t^{inc} = F_t.$$

也就是说会换掉 bit-exact 目标。

当前你们既然已有测试名：

decisive_endpoint_tower_parity_longhistory

并要求增量与全量 bit-exact，那么 canonical 已经选择了：

F_t = 全量重算塔.

因此：

未确认 frontier 视为不可变就是 bug。

八、正确增量算法

定义 seal 谓词：

$$Seal(w, t) = 1$$

表示窗口 w 在时刻 t 后不会被未来 bar 改写。

定义最后已封存边界：

$$b_t = \max\{end(w) : Seal(w, t) = 1\}.$$

增量更新应为：

$$T_{t+1}^{inc} = Prefix(T_t^{inc}, b_t) \oplus Detect(h_{b_t:t+1}).$$

其中：

$$Prefix(T_t^{inc}, b_t)$$

只保留所有完全结束于 b_t 之前的 sealed 结构。

所有从 b_t 之后开始的结构，包含最后未确认窗口，都必须重算。

如果无法证明某窗口 sealed，就必须把它放进 frontier。

更保守的写法

若 seal 难以定义，直接选：

$$b_t = \text{last stable endpoint before current active candidate.}$$

也就是：

rollback 到最早可能受未来 bar 影响的候选起点。

性能差一些，但保证 bit-exact。

九、增量等价定理

定理

假设：

1. 全量检测函数 F 确定；
2. 存在边界 b_t ，使得对所有未来扩展：

$$Prefix(F(h_{0:t}), b_t) = Prefix(F(h_{0:t+1}), b_t).$$

即 b_t 前 prefix 稳定；

3. 增量算法使用：

$$T_{t+1}^{inc} = Prefix(T_t^{inc}, b_t) \oplus Detect(h_{b_t:t+1});$$

4. 初始：

$$T_0^{inc} = F(h_0).$$

则：

$$T_t^{inc} = F(h_{0:t}) \quad \forall t.$$

证明

归纳。

基例成立。

假设：

$$T_t^{inc} = F(h_{0:t}).$$

由 prefix stability：

$$Prefix(T_t^{inc}, b_t) = Prefix(F(h_{0:t}), b_t) = Prefix(F(h_{0:t+1}), b_t).$$

后缀部分由同一个确定性检测函数对同一历史片段：

$$h_{b_t:t+1}$$

计算，所以与全量重算后缀一致。

因此：

$$T_{t+1}^{inc} = F(h_{0:t+1}).$$

证毕。

当前实现为什么违反定理？

它选的 b_t 太晚，把未确认最后窗口放进 prefix。

于是 prefix stability 不成立：

$$\text{Prefix}(F(h_{0:t}), b_t) \neq \text{Prefix}(F(h_{0:t+1}), b_t).$$

所以归纳证明断裂。

十、问题②最终裁决

在“增量必须等于全量重算”的 bit-exact 目标下，这是实现 bug，应该修。

正确修法：

增量 resume 只能从 sealed prefix 之后开始；未确认 frontier 必须重算。

如果你们不修，高级别计数与候选都会被污染：

$$\Gamma_{\ell \geq 2}^{inc} \neq \Gamma_{\ell \geq 2}^{full}.$$

于是：

$$\hat{\mu}_{\ell \geq 2}^{inc}$$

不是目标估计量。

因此：

所有“高级别无 alpha”的结论都必须撤回或标注为 frontier-bug 污染。

十一、两个问题的共同数学结构

两个问题都是同一种错误：

把局部边界条件误当成全局结构语义。

问题①：

$$\text{end_index} = \text{source}$$

是局部端点挂靠关系，不是区间套跨级递归关系。

正确全局语义是：

$$J_{child} \subseteq J_{parent}.$$

问题②：

“最后窗口已经登记”是局部缓存状态，不是全局确认状态。

正确全局语义是：

只有 sealed prefix 可复用，mutable frontier 必须重算。

所以两个都应修。

十二、验收测试建议

问题①测试

1. 构造三层嵌套：

$$J_0 \subset J_1 \subset J_2$$

但：

$$\text{end}(J_1) \neq \text{source}(J_0).$$

旧实现应失败，新实现应成功。

2. 构造边界等号场景：

$$\text{source} = \text{end}(m).$$

新实现应与旧实现一致。

3. 构造多个包含父候选：

$$J_{child} \subset J_a, \quad J_{child} \subset J_b.$$

确认 Sel 唯一且 bit-exact。

4. 对每个成功证书断言：

$$J_e \subseteq \dots \subseteq J_\ell.$$

问题②测试

1. 每 bar 增量塔与全量塔 bit-exact，而不仅是最终相等。
2. 构造最后未确认中枢被新 bar 修改的序列。
3. 确认旧实现多一个中枢，新实现与全量一致。
4. 长历史 CL/BTC 15万/30万 bar parity 全绿。

最终总裁决

问题①：必须把跨级 rung 定位从端点相等改为区间包含，且最好以 $J_{child} \subseteq J_{parent}$ 为实

唯一性由良式分解或 Sel_Θ 保证。

问题②：若 full recompute 是 canonical，则这是实现 bug；必须重算未确认 frontier。

不修这两个问题，就不能声称已经回测了缠师 100% 区间套多级判据。

这两个修复都是 bit-exact 核心定义级改动，但它们不是 workaround；它们是把实现从“简化边界规则”拉回到真正的递归区间套和 canonical 全量塔语义。



